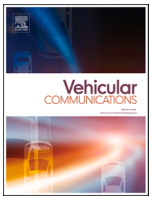
Vehicular Communications 60 (2026) 101031



Contents lists available at ScienceDirect

Vehicular Communications

journal homepage: www.elsevier.com/locate/vehcom





Energy-aware joint task assignment and 3D path planning for cooperative multi-UAV networks ⋆

Yijun Lu [a], Yufang Liu [b], Yuyin Ma [b], Tingting Xiao [c], Pengpeng Qiao [d,*]

[a] School of Computer Science and Engineering, Waseda University, Tokyo, Japan
[b] School of Software, Xinjiang University, Urumqi, Xinjiang, China
[c] School of Computer Science, Hangzhou Normal University, Hangzhou, Zhejiang, China
[d] School of Computing, Institute of Science Tokyo, Tokyo, Japan

ARTICLE INFO

Keywords:
UAV-assisted networks
Vehicular communications
Cooperative sensing
Task assignment
3D path planning
Energy efficiency
SE-ABC algorithm

ABSTRACT

Unmanned Aerial Vehicle (UAV) networks have emerged as a pivotal solution for mobile sensing tasks in complex civilian and military environments. Compared with single-UAV systems, multi-UAV collaboration offers significant advantages, including superior robustness against single-point failures and enhanced efficiency in multi-target sensing. However, coordinating such networks in 3D environments with known threats presents significant challenges, particularly due to the complex coupling between high-level task allocation and low-level trajectory planning. To address these challenges, we propose a hierarchical collaborative optimization framework that effectively decouples the spatio-temporal constraints into two interlinked layers. First, we introduce an extended Hungarian algorithm tailored for asymmetric costs to ensure global optimality in task allocation. Second, we develop a novel bio-inspired Self-Evolved Artificial Bee Colony (SE-ABC) algorithm to generate safe, energy-efficient 3D trajectories dynamically. In contrast to many recent hybrid or learning-based planners that typically rely on offline training and extensive parameter tuning, SE-ABC offers a training-free and lightweight evolutionary search with self-adaptive operators, enabling online trajectory refinement under coupled energy–threat costs while remaining easy to implement and interpret. We validated the proposed framework using a DJI Phantom 3 quadcopter prototype and extensive simulations. Experimental results demonstrate that our approach significantly outperforms existing benchmarks, such as Particle Swarm Optimization (PSO), Genetic Algorithm (GA), and the standard Artificial Bee Colony (ABC). On average, the proposed method reduces the total mission cost by 15% (accounting for safety), shortens path length by 23%, and decreases task completion time by 25%.

1. Introduction

Unmanned aerial vehicle (UAV), a.k.a. drone, has been widely used for mobile sensing in civilian and military applications, such as surveillance [1], reconnaissance [2], and video- or photo-graphy [3]. These applications typically require the UAV to traverse, hover, and collect data at a number of observation locations in 3D space. Recently, researchers have increasingly focused on multi-UAV network systems. Compared with a single-UAV system, a multi-UAV network offers significant advantages, including robust multi-target sensing capabilities, resilience to single-point failures, and superior energy efficiency, as extensively discussed in recent studies [4,5].

However, realizing the full potential of multi-UAV collaboration in complex 3D environments remains a formidable challenge. The core difficulty lies in the efficient coordination of resources to minimize global costs while ensuring safety. Specifically, the mission of mobile sensing comprises two tightly coupled sub-problems: *task assignment* (allocating targets to UAVs) and *path planning* (generating safe trajectories).

To address these problems, extensive research has been conducted. For task assignment, heuristic algorithms such as simulated annealing [6], ant colony optimization (ACO) [7], and genetic algorithms (GA) [8] are commonly used. While these methods are flexible, they often suffer from slow convergence speed and instability when dealing with large-scale or real-time dynamic changes. Exact methods like the Hungarian algorithm [9,10] offer optimal solutions but traditionally fail in asymmetric scenarios where the number of tasks and agents differs, a common occurrence in real-world missions [11]. Similarly, for 3D path planning, swarm intelligence algorithms like Particle Swarm

⋆ This work was supported by JST BOOST, Japan (Grant Number JPMJBS2429).
* Corresponding author.
E-mail addresses: yijun@ruri.waseda.jp (Y. Lu), liuyufang@stu.xju.edu.cn (Y. Liu), mayuyin@xju.edu.cn (Y. Ma), ttxiao_1@hznu.edu.cn (T. Xiao), pengpeng.q.860e@m.isct.ac.jp (P. Qiao).

https://doi.org/10.1016/j.vehcom.2026.101031
Received 16 December 2025; Received in revised form 2 April 2026; Accepted 14 April 2026
Available online 25 April 2026
2214-2096/© 2026 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

Optimization (PSO) [12] and Artificial Bee Colony (ABC) [13] have gained popularity due to their simplicity. However, these standard algorithms frequently overlook the balance between exploration and exploitation, leading to premature convergence and entrapment in local optima, especially in complex environments with dense obstacles.

More critically, most existing approaches treat task assignment and path planning as isolated sequential processes or ignore the dynamic coupling between them. They often assume static environments and symmetric resource availability, failing to account for real-time uncertainties such as sudden battery depletion or asymmetric task loads. This disconnect often results in sub-optimal system performance, where a task is assigned to a UAV that cannot find a feasible or efficient path to execute it due to unmodeled constraints.

In this paper, we address these gaps by formulating the multi-UAV sensing mission as a joint optimization problem that explicitly considers dynamic constraints and asymmetric costs in 3D space. Unlike previous works that rely on standard heuristics, we propose a hierarchical collaborative framework that integrates a rigorous assignment strategy with a robust path planning mechanism. Beyond presenting two algorithmic components, our novelty lies in the framework-level integration: we decouple globally optimal asymmetric assignment from threat–energy-aware 3D trajectory generation, and specify an explicit interface that updates the assignment when mission costs change dynamically. This design enables consistent coordination between task allocation and trajectory feasibility in threat-aware environments.

Compared with existing joint or sequential optimization frameworks, the proposed method is distinguished by three tightly coupled features at the framework level. First, the task-assignment layer explicitly supports asymmetric task–UAV settings through an extended Hungarian algorithm, rather than relying on symmetric matching assumptions or purely heuristic allocation. Second, the interaction between the manager and UAV layers is designed as an event-triggered update–reassign–replan loop, allowing the framework to respond promptly to dynamic mission-cost changes such as battery depletion. Third, the trajectory layer optimizes an explicit threat–energy-aware objective in 3D space via the SE-ABC algorithm, so that task feasibility and actual execution cost remain consistently coordinated. These features differentiate our framework from prior approaches that either optimize assignment and planning sequentially or only partially account for online dynamics, asymmetric workloads, and threat-aware trajectory generation.

The main contributions of this work are summarized as follows:

1. We propose a hierarchical collaborative optimization framework that decouples task allocation and path planning. This includes an extended Hungarian algorithm that preserves global optimality under asymmetric assignment and supports dynamic updates under battery-induced cost changes.
2. We develop a bio-inspired SE-ABC algorithm for safe and energy-efficient 3D trajectory generation. To balance exploration and exploitation, we incorporate a self-adaptive guidance strategy, a Boltzmann-based probabilistic selection scheme, and a chaotic local search operator.
3. We validate the proposed approach through extensive simulations and a DJI Phantom 3 prototype. The results show that our framework outperforms standard baselines (PSO, ACO, and GA), reducing the total mission cost by 15% (including safety cost), shortening path length by 23%, and decreasing task completion time by 25%.

The rest of this paper is organized as follows. Section 2 reviews the related work. Section 3 presents the multi-UAV network model and problem formulation. Sections 4 and 5 detail the strategies for task assignment and the proposed SE-ABC algorithm, respectively. We evaluate the performance in Section 6 and conclude the paper in Section 7.

2. Related work

The rapid evolution of Multi-UAV networks has catalyzed advancements in diverse applications, from wireless communication relaying [14] and mobile edge computing [15] to automated surveying systems [16,17]. Efficiently coordinating these networks requires robust optimization across multiple layers. At the network layer, recent state-of-the-art protocols, including lightweight trustworthy message exchange frameworks [18] and Intelligent Clustering Routing Approaches (ICRA) [19], excel at ensuring secure transmission and routing efficiency. Building on such reliable communication infrastructures, our proposed framework operates at the complementary application layer to jointly optimize physical 3D spatial trajectories and task allocations. This section reviews existing strategies in task assignment and path planning, highlighting the specific limitations that our framework aims to address. To better position our contributions, Table 1 provides a comparative summary contrasting representative existing studies with our hierarchical approach.

2.1. Task assignment strategies

Task assignment is the cornerstone of multi-UAV mission planning. Existing approaches generally fall into heuristic-based methods and exact algorithms.

Heuristic algorithms are widely favored for their adaptability to nonlinear constraints. For instance, Simulated Annealing (SA) has been applied to complex logistics and routing problems [20,21]. Bio-inspired methods like Ant Colony Optimization (ACO) have shown promise in dynamic task distribution [22,23], while Genetic Algorithms (GA) are extensively used for large-scale scheduling optimization [24,25]. However, these population-based heuristics often suffer from instability and

Table 1
Comparative summary of representative studies and our framework.

Work	Task Assignment	Path Planning	Joint / Hierarchical	Asymmetric Tasks	Energy/Threat-Aware	Online Dynamics
Heuristic assignment (SA/ACO/GA) [20,22,24]	Heuristic	–	×	✓	partial	partial
Exact matching (Hungarian variants) [9–11]	Exact	–	×	limited	partial	partial
SI-based 3D planners (PSO/ACO/ABC) [12,13,39]	–	SI / Metaheuristic	×	–	partial	partial
DRL-based planners [43]	– / partial	Learning-based	×	–	partial	✓
Joint / sequential frameworks [44,45]	Mixed	Mixed	✓ (often sequential)	partial	partial	partial
This work	**Extended Hungarian**	**SE-ABC**	✓ (hierarchical)	✓	✓	✓ (battery-triggered)

slow convergence rates, making them prone to local optima in high-dimensional solution spaces [26].

Conversely, exact methods, particularly the Hungarian algorithm, guarantee optimal solutions. This method has been successfully adapted for various matching problems, including 3D object detection [27,28] and energy resource scheduling [29]. Nevertheless, the standard Hungarian algorithm is strictly limited to symmetric assignment (one-to-one mapping). Although recent studies have explored dynamic assignment under uncertainties [30,31] and consensus-based heterogeneous allocation [11,32], seamlessly adapting exact methods to asymmetric, energy-constrained missions without excessive computational overhead remains an open issue.

2.2. UAV path planning

For 3D trajectory generation, Swarm Intelligence (SI) has become dominant over traditional methods like motion capture-based planning [33]. Recent years have seen the emergence of novel evolutionary strategies, such as the Red-billed Blue Magpie Optimizer [34] and stochastic frameworks for autonomous navigation [35,36]. Among these, Particle Swarm Optimization (PSO) and its variants are widely adopted, with recent improvements focusing on multi-sensor fusion [37] and adaptive position updating [12]. Similarly, Ant Colony based planners have been optimized for urban rescue scenarios [38,39].

Despite these advances, standard swarm algorithms, including the Artificial Bee Colony (ABC) [13,40], frequently struggle to balance exploration and exploitation. This imbalance often leads to premature convergence in complex environments with dense obstacles [41,42]. While advanced methods like Deep Reinforcement Learning (DRL) [43] offer robustness, they often require substantial computational resources. From a performance–complexity perspective, DRL planners can be highly adaptive after training but typically require an offline training pipeline and non-trivial tuning/retraining as conditions change. In contrast, our training-free SE-ABC directly optimizes an explicit threat–energy-aware objective with predictable computational overhead, making it suitable for the considered settings.

Crucially, most existing studies treat task assignment and path planning as decoupled, sequential problems [44,45]. They often optimize assignment based on simple Euclidean metrics, ignoring that actual flight costs depend on 3D obstacle avoidance [46] and communication constraints [47]. Even approaches utilizing neural networks [48] rarely address the real-time coupling between asymmetric task loads and 3D trajectory feasibility. Our work bridges this gap by integrating an extended Hungarian algorithm with a self-evolved ABC planner in a unified hierarchical framework.

3. Problem formulation

In this section, we start with the definition of a multi-UAV sensing system which consists of two main components, namely the task assignment and path planning.

3.1. The multi-UAV sensing system

We consider the terrestrial base station as the system manager, and individual UAVs as agents. A motivating scenario of such a system is illustrated in Fig. 1. The manager obtains the information regarding a set of target locations, a number of UAVs, and the threats in the space, to form a holistic multi-UAV network. The system consists of two components: (1) the manager executes the task assignment; and (2) each UAV acts as an independent individual to execute the path planning. For clarity, Fig. 2 summarizes the overall hierarchical workflow and the battery-triggered dynamic reassignment loop.

Task assignment: Each of the target locations needs to be covered by the planned path of the UAVs in the system. Based on the set of target locations, the manager creates multiple sensing tasks, where each task

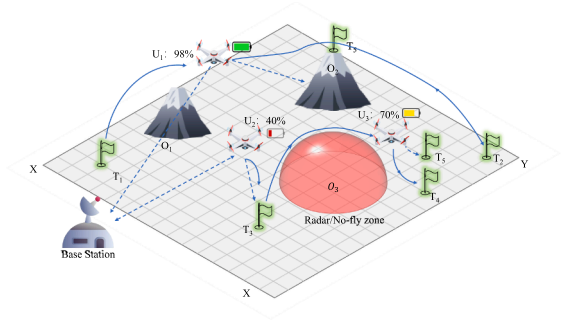


Fig. 1. Illustration of a motivating scenario for multi-UAV mobile sensing.

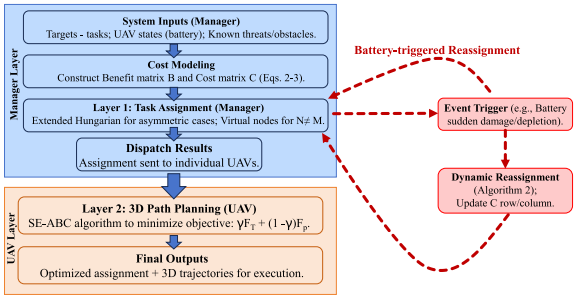


Fig. 2. Overview of the hierarchical workflow.

consists of a series of target locations. Then, the manager determines the assignment strategy, i.e., allocating tasks to multiple UAVs in the system, such that all target locations are covered efficiently.

Path planning: After receiving the task assignment results, each UAV adopts an appropriate path planning algorithm to obtain a safe and feasible path that traverses all target locations defined in its assigned task, subject to specific flight constraints.

Layer interaction and reassignment trigger. The system operates on an event-triggered *update–reassign–replan* loop. Upon any dynamic cost change (e.g., battery depletion), the manager updates the assignment via Algorithm 2 and dispatches it to the UAVs, which then re-run SE-ABC to adapt their 3D trajectories. While primarily focused on internal system dynamics, this architecture naturally accommodates external uncertainties. For instance, dynamic moving threats can be managed by updating the threat degree D_k (Eq. (9)) in real-time. This directly alters the cost matrix C, instantly triggering the same reassignment loop. Furthermore, communication delays between the manager and UAVs can be seamlessly mitigated by incorporating predictive state bounding or time-stamped synchronization into the cost modeling phase.

From a practical deployment perspective, the proposed framework can tolerate moderate communication latency and asynchronous state reporting between the manager and UAV agents. If delayed or partial system information is received, the manager performs reassignment using the most recently available synchronized state estimate, and the updated assignment is dispatched once fresher information becomes available. Meanwhile, each UAV continues to execute its latest feasible local trajectory until the new assignment is received, which helps preserve mission continuity. Therefore, the framework does not rely on strict instantaneous global synchronization; instead, it follows an event-triggered update mechanism that remains applicable under imperfect communication conditions, although large delays may reduce the responsiveness of global replanning.

Overall, the proposed methodology can be understood as a two-layer decision process. At the system level, the manager first estimates the benefit and cost associated with each task–UAV pair and determines the assignment, including asymmetric cases and dynamic updates when mission costs change. At the execution level, each UAV then generates

a feasible 3D trajectory for its assigned task by minimizing a weighted objective that balances threat avoidance and energy consumption. In this sense, the assignment layer determines which UAV should perform which task, whereas the trajectory layer determines how the assigned task should be executed safely and efficiently in 3D space. The mathematical formulations in the following sections formalize these two roles and their interaction within the proposed hierarchical framework.

3.2. The task assignment problem

The objective of task assignment is to maximize the overall mission efficiency by balancing the task benefits against the operational costs. We consider the battery life constraint of each UAV and aim to assign lower-cost tasks to UAVs with lower battery levels, while prioritizing high-value tasks.

Let $\{U_1, U_2, \ldots, U_n\}$ denote the set of N UAVs, and $\{T_1, T_2, \ldots, T_m\}$ denote the set of M tasks. The task assignment matrix is defined as:

$$A_{ij} = \begin{cases} 1, & \text{if } U_i \text{ is assigned task } T_j, \\ 0, & \text{otherwise.} \end{cases} \tag{1}$$

Based on the task assignment matrix, we define the benefit matrix B and the cost matrix C. The corresponding total benefit of assignment A is:

$$B(A) = \sum_{i=1}^{n} \sum_{j=1}^{m} A_{ij} b_{ij}. \tag{2}$$

The total cost of A is:

$$C(A) = \sum_{i=1}^{n} \sum_{j=1}^{m} A_{ij} c_{ij}, \tag{3}$$

where b_{ij} represents the benefit (e.g., priority or sensing value), and c_{ij} is the cost (e.g., energy consumption) when UAV i performs Task j.

To achieve the optimal trade-off, we seek to maximize the benefit-to-cost ratio of the task assignment. Hence, the task assignment problem can be formulated as:

$$\max \frac{B(A)}{C(A)}, \tag{4}$$

$$s.t. \sum_{i=1}^{n} A_{ij} = 1, \quad \sum_{j=1}^{m} A_{ij} = 1. \tag{5}$$

where the first constraint ensures that each task is assigned to exactly one UAV, and the second constraint ensures that each UAV is assigned to exactly one task. Note that this formulation represents a standard one-to-one assignment problem; asymmetric cases (where $N \neq M$) are handled by introducing virtual nodes as detailed in Section 4.

3.3. The path planning problem

UAV path planning aims to find a reasonably optimized flight path in a specific space to meet various restrictions on UAV flight. We formulate the problem as a combination of the minimum dominating path problem and the constrained minimum dominating set problem.

Compared with path planning in a 2D plane, the problem in a 3D space has more practical applications in the real world. In this work, the starting location and the target location are defined as P_s and P_t, respectively. There are threats (or obstacles) in the space, such as radars, mountains, or tall buildings. Assume there are k obstacles with known locations in the space, namely, $O_1, O_2, \ldots, O_k$. UAV path planning refers to finding the best flight path for a safe arrival at the destination, avoiding the flight range of any known obstacle O_i.

Location coordinates in 3D space: Fig. 3(a) illustrates a cubic region defined by eight vertices ABCD-EFGH in a 3D coordinate system, where A is the starting location P_s, and G is the target location P_t. Suppose the path between A and G is composed of n_{sum} locations. We divide

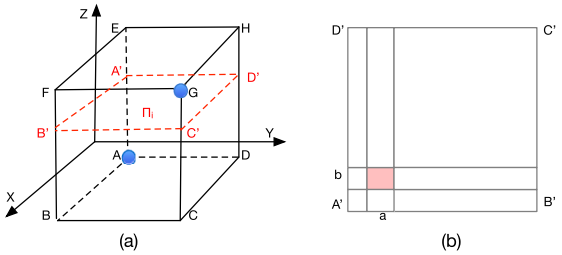


Fig. 3. (a) A three-dimensional space for path planning, where A is the starting location P_s and G is the target location P_t ; (b) the plane $A'B'C'D'$ is divided into a number of grids.

the space into n_{sum} planes of equal size along the Z-axis, such that each of the n_{sum} locations on the path belongs to a specific divided plane.

Let $\Pi_i (i = 1, 2, \ldots, n_{sum})$ be one of the planes. Given any plane, we can further divide it into grids with a side length of 1, as shown in Fig. 3(b). Let $a = |AB|/l$ and $b = |AD|/l$. Consequently, the plane $A'B'C'D'$ is divided into $a \cdot b$ grids. Any discrete grid's coordinates can be represented by $P_i(x_i, y_i, z_i)$. Then, we plan the path to traverse a sequence of grids in this 3D coordinate system. The length L of the path can be written as:

$$L = \sum_{i=1}^{n_{sum}-1} \sqrt{(x_{i+1} - x_i)^2 + (y_{i+1} - y_i)^2 + (z_{i+1} - z_i)^2}, \tag{6}$$

where (x_i, y_i, z_i) are the coordinates of the ith location on the path.

Path cost under threats: The threat cost f_T and the power consumption cost f_P are taken into consideration. We define:

$$F_T = \int_0^L f_T dl, \quad F_P = \int_0^L f_P dl. \tag{7}$$

The path planning problem is formulated as:

$$\min \gamma F_T + (1 - \gamma) F_P, \tag{8}$$

where $\gamma \in [0, 1]$ is a weighting parameter. In this study, we use $\gamma = 0.5$, indicating that threat avoidance is as important as power conservation. Assuming the UAV velocity is constant, the power cost is proportional to the path length L.

Threat cost calculation: Given any two adjacent locations P_i and P_{i+1} on a path, we define the segment (P_i, P_{i+1}) as a subpath. The threat cost over this subpath is approximated using five discrete sample locations. These locations are positioned at distances of 1/10, 3/10, 5/10, 7/10, and 9/10 of the subpath length, as shown in Fig. 4. If any location on the subpath is within the safety range of a threat, the threat cost is calculated as:

$$f_{i,P_iP_{i+1}} = \frac{L_i}{5} \sum_{k=1}^{N_k} D_k \sum_{s \in \{0.1, \ldots, 0.9\}} \frac{1}{d_{s,i,k}^4}, \tag{9}$$

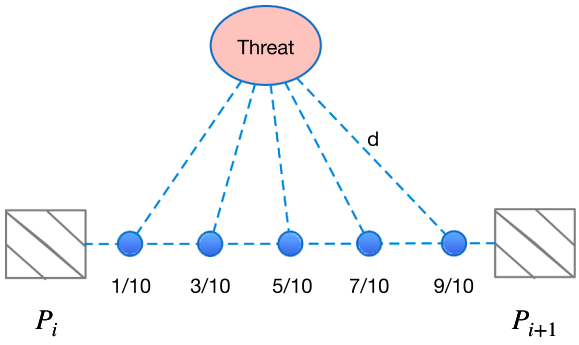


Fig. 4. Computation of threat cost by an approximation based on five discrete points between P_i and P_{i+1}).

Table 2
Summary of key mathematical notations.

Symbol	Meaning
$\mathcal{G} = \{\mathcal{V}, \mathcal{U}, \mathcal{E}\}$	Bipartite graph for assignment (tasks vs. UAVs).
n, m	Number of tasks and UAVs (i.e., $\lvert\mathcal{V}\rvert = n$, $\lvert\mathcal{U}\rvert = m$).
T_i, U_j	Task node i and UAV node j.
c_{ij}	Assignment edge cost between T_i and U_j (cost matrix entry).
A	A (perfect/complete) assignment (matching) in the bipartite graph.
$C(A)$	Total cost of an assignment A, e.g., $C(A) = \sum_{(i,j)\in A} c_{ij}$.
α_i, β_j	Dual variables (vertex labels) in the Hungarian method, satisfying $\alpha_i + \beta_j \le c_{ij}$.
I^*, J^*	Index sets of task/UAV nodes in the Hungarian tree (used in dual updates).
θ	Step size for updating dual variables in Eq. (12).
t	Current iteration index in SE-ABC (used instead of "cycle").
MaxCycle	Maximum number of iterations in SE-ABC.
SN	Population size (number of food sources) in ABC/SE-ABC.
ξ	Self-adaptive coefficient in the guidance strategy (Eq. (17)).
T, T_0	Boltzmann temperature and initial temperature in Eq. (18).

where N_k is the number of threats, L_i is the length of the subpath, D_k denotes the degree of threat, and $d_{s,i,k}$ denotes the distance from the sth sample point (e.g., $s = 1/10$) to threat k. Here, D_k is a configurable parameter that encodes the relative hazard level of threat k; it can be adjusted to reflect different safety margins or mission priorities in different environments. To approximate the threat cost on (P_i, P_{i+1}), we sample five locations at $s \in \{0.1, 0.3, 0.5, 0.7, 0.9\}$ of the subpath length from P_i, as shown in Fig. 4. If any sampled location lies within the safety range of a threat, we compute the threat cost as

$$f_{i,P_iP_{i+1}} = \frac{L_i}{5}\sum_{k=1}^{N_k} D_k\Big(\frac{1}{d^4_{0.1,i,k}} + \frac{1}{d^4_{0.3,i,k}} + \frac{1}{d^4_{0.5,i,k}} + \frac{1}{d^4_{0.7,i,k}} + \frac{1}{d^4_{0.9,i,k}}\Big), \tag{10}$$

where N_k represents the number of threats in vicinity, L_i refers to the length of the subpath (P_i, P_{i+1}), D_k denotes the degree of threats that is an integer predefined by the government [49], and $d_{0.1,i,k}$ (or $d_{0.3,i,k}$, $d_{0.5,i,k}$, $d_{0.7,i,k}$, $d_{0.9,i,k}$) denotes the distance from the 1/10 (or 3/10, 5/10, 7/10, 9/10) location on the subpath to the threat location.

In Section 5, we use the artificial bee colony algorithm to find the optimal path that minimizes F.

For readability, we provide a consolidated notation table in Table 2 summarizing frequently used symbols throughout the paper.

4. Asymmetric task assignment with dynamic costs

In this section, we **extend** the Hungarian algorithm to address the problem of assigning multiple sensing tasks to multiple UAVs, when the number of tasks and the number of UAVs are not equal, and meanwhile, the cost is dynamic.

4.1. Three cases of the problem

Given a bipartite graph, $G = \{T, U, E\}$ as illustrated in Fig. 5(a), T and U are the sets of nodes in each partition of the graph, and E is the set of edges. The task assignment matrix, c_{ij}, represents the cost of the edge ij in G when task T_i is assigned to UAV U_j; it is ∞ when there is no edge ij in G, as shown in Fig. 5(b).

Symmetric vs. asymmetric case: The original Hungarian algorithm assumes the *symmetric case*— the number of tasks is equal to the number of agents (i.e., UAVs in this work), and a task can only be performed by one agent. Yet our problem considers the *asymmetric case*—the number of tasks and the number of UAVs are not equal in general, and a task is usually solved through a collaboration among multiple UAVs, or a UAV performs multiple tasks.

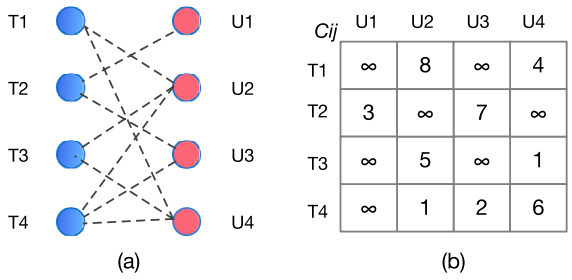


Fig. 5. (a) A bipartite graph, where the blue circle represents the set of Tasks and the red circle represents the set of UAVs; (b) An alternative matrix representation showing the edge cost, where each missing edge is given a large cost. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Dynamic cost due to the change of battery life: Another important factor that affects the task assignment is the dynamic cost due to the change of battery life. For example, in a multi-UAV network, the cost of traversing a path may change when the battery life of a certain UAV has dropped significantly, which indicates that a new task assignment needs to be made.

Therefore, the task assignment problem in reality can be addressed in the following three cases.

- **Symmetric assignment:** Assign n tasks to n UAVs with the minimum total cost, where task i is assigned to UAV j with a non-negative cost c_{ij}.
- **Asymmetric assignment:** Assign n tasks to m UAVs ($n \neq m$) with the minimum total cost. We need to find an optimal assignment, given the $n \times m$ cost matrix.
- **Asymmetric assignment with dynamic cost:** The assignment problem with changing costs, in which we are given an optimal solution to an $n \times m$ assignment problem and a row or column of changed edge costs. The objective is to efficiently find an optimal assignment to the new problem with changed costs.

Next, we discuss the second and the third cases respectively.

4.2. Asymmetric assignment

Start from symmetric assignment: The following symmetric assignment problem can be solved by the conventional Hungarian algorithm.

$$\begin{aligned} \min \quad & Cost = \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}c_{ij}, \\ s.t. \quad & \sum_{i=1}^{n} A_{ij} = 1 \quad (j = 1, 2, \dots, n), \\ & \sum_{j=1}^{n} A_{ij} = 1 \quad (i = 1, 2, \dots, n), \\ & A_{ij} \in \{0, 1\}. \end{aligned} \tag{11}$$

Lemma 1. *If a value is added to or subtracted from all of the entries of any one row or column of a cost matrix C, then the optimal assignment for the resulting cost matrix C' is also an optimal assignment for the original cost matrix C.*

Proof. Assume that x_i is subtracted from all of the entries of row i, and y_j is subtracted from all of the entries of column j from the

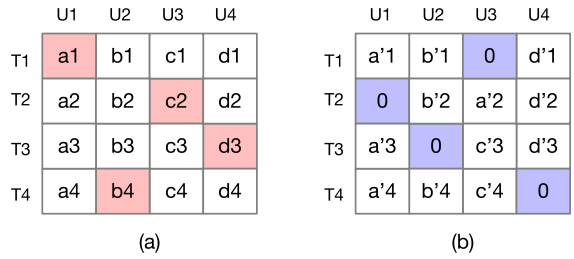


Fig. 6. (a) Original cost matrix C, where orange grids represent the smallest cost in each row; (b) final cost matrix C', where blue grids represent that the tasks are assigned to the corresponding UAVs. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

original cost matrix C. Then after the conversion, the cost matrix is $C' = \sum_{i=1}^{n}\sum_{j=1}^{n}(c_{ij} - x_i - y_j) \times A_{ij}$.

Then $C' = \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}c_{ij} - \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}x_i - \sum_{i=1}^{n}\sum_{j=1}^{n} A_{ij}y_j$. Because $\sum_{i=1}^{n} A_{ij} = 1$ and $\sum_{j=1}^{n} A_{ij} = 1$, the final result of cost matrix C' is $C - \sum_{i=1}^{n} x_i - \sum_{j=1}^{n} y_j$.

Since $\sum_{i=1}^{n} x_i + \sum_{j=1}^{n} y_j$ is a constant number, minimizing C' is equivalent to minimizing C. Thus, the optimal assignment for the resulting cost matrix C' is also an optimal assignment for the original cost matrix C. □

Suppose we have four UAVs and four different tasks to be assigned, as shown in Fig. 6. The problem is to find the lowest-cost way to assign the tasks. The orange grids shown in Fig. 6(a) represent the minimum of the cost matrix in each row (assuming a_1, b_4, c_2, d_3 are the minimums). After applying the Hungarian algorithm, we obtain another matrix in Fig. 6(b), where the values of blue grids become **zero**. This matrix C' has a zero for each row, and these zero-values are also distributed in different columns. Thus, the least-cost assignment is obtained: assign Task 1 to UAV 3, Task 2 to UAV 1, Task 3 to UAV 2, and Task 4 to UAV 4.

An improved Hungarian algorithm for addressing the asymmetric assignment: When $n \neq m$ in an $n \times m$ cost matrix, we need to perform a matrix transformation and then apply an improved Hungarian algorithm to address the asymmetric assignment problem.

Theorem 1. *(Optimality preservation under augmentation). Consider an $n \times m$ cost matrix with a one-to-one assignment formulation and additive costs. If it is augmented to a square $k \times k$ matrix ($k = \max(n, m)$) by adding $|n - m|$ virtual rows or columns that represent dummy tasks (idle UAVs) or dummy UAVs (unassigned tasks), then any optimal assignment of the augmented problem, after removing the dummy matches, yields an optimal assignment for the original asymmetric problem.*

Proof. Let $k = \max(n, m)$ and let $C'' \in \mathbb{R}^{k \times k}$ denote the augmented matrix. The augmented problem minimizes

$$\sum_{i=1}^{k}\sum_{j=1}^{k} A_{ij}c''_{ij}$$

subject to $\sum_{i=1}^{k} A_{ij} = 1$, $\sum_{j=1}^{k} A_{ij} = 1$, and $A_{ij} \in \{0, 1\}$, where dummy matches correspond to assignments involving virtual nodes. Because dummy matches do not correspond to real task–UAV pairs in the original problem, removing them from an optimal solution of the augmented problem produces a feasible assignment for the original asymmetric case. Moreover, under the additive cost formulation, the total cost contributed by real task–UAV pairs is preserved, and any improvement in the original asymmetric assignment would induce a lower-cost feasible solution in the augmented problem, contradicting optimality. Therefore, the reduced assignment is optimal for the original problem. □

The above augmentation assumes a standard one-to-one matching formulation with additive costs, where virtual rows/columns encode dummy matches (idle UAVs when $m > n$, or dummy UAVs for remaining tasks when $n > m$). The final assignment is obtained by discarding the dummy matches. By transforming the $n \times m$ matrix into a square matrix by adding virtual entries (UAVs or Tasks) with zero values, we perform the Hungarian algorithm on the augmented cost matrix, and then remove the virtual entries. The pseudo code is given in Algorithm 1.

Algorithm 1 The asymmetric assignment algorithm.

Require: A bipartite graph, $\{V, U, E\}$ with an $n \times m$ matrix of edge costs.
Ensure: A complete assignment.
1: Construct a square cost matrix by adding virtual rows or columns with zero entries to handle the asymmetry ($|V| \neq |U|$).
2: For each row, subtract the minimum row value from all entries in that row.
3: For each column, subtract the minimum column value from all entries in that column.
4: Execute the conventional Hungarian algorithm on the square cost matrix.
5: Delete the assignments associated with virtual UAVs or Tasks.
6: Output the resulting assignment.

Computational complexity. After augmentation, the assignment is solved on a square matrix of size $k = \max(n, m)$. The Hungarian algorithm runs in polynomial time, typically $O(k^3)$, and therefore the assignment-layer overhead scales cubically with the larger of the task/UAV counts. This provides a tractable computational bound when the number of UAVs and tasks increases.

4.3. Dynamic assignment

In this case, we are given an optimal solution to an $n \times m$ task assignment problem, and a row or a column of changed costs in the cost matrix. For example, one UAV's battery life suddenly drops, or a new UAV is added into the network, which can lead to changed values in the cost matrix. The objective here is to efficiently find an optimal solution to the new problem under changed costs.

The algorithm assigns dual variables α_i and β_j to each task (node) T_i and each UAV (node) U_j, respectively. It exploits the fact that the dual of the minimization version of the assignment problem is feasible when $\alpha_i + \beta_j \leq c_{ij}$. The Hungarian algorithm maintains feasible values for all the values of α_i and β_j from the initialization through the termination.

- **A**dmissible edges: An edge in the bipartite graph is called admissible when $\alpha_i + \beta_j = c_{ij}$.
- **E**quality subgraph: The subgraph consisting of only the currently admissible edges is called the equality subgraph.
- **A**ugmenting path: If P is the path of two unassigned vertices, and the assigned edges and unassigned edges alternately appear on P, then P is an augmented path.
- **H**ungarian tree: Searching for an augmenting path in a graph involves these alternating paths in a breadth-first manner, and the process can be called growing a Hungarian tree.

Theorem 2. *Let G' be the equality subgraph of G. If G' contains a perfect assignment A', then A' is an optimal assignment of G.*

Proof. In this paper, a feasible vertex labeling is defined in terms of the costs, $\alpha_i + \beta_j \leq c_{ij}$.

Suppose that G' contains a perfect assignment A'. Since A' is the subgraph of G, A' is also a perfect assignment for G. Therefore, the cost of this assignment is $C(A') = \sum_{(i,j \in A')} c_{ij} = \sum \alpha_i + \sum \beta_j$. However, for any perfect assignment A of G which requires that $\alpha_i + \beta_j \leq c_{ij}$, the cost of A is $C(A) = \sum_{(i,j \in A)} c_{ij} \geq \sum \alpha_i + \sum \beta_j$. The conclusion is drawn that $C(A') \leq C(A)$, and thus $C(A')$ is an optimal assignment. □

The basic idea of the Hungarian algorithm is to repeatedly search for augmenting paths in the equality subgraph. If an augmenting path is found, the current set of assignment is augmented by flipping the assigned and unassigned edges along this path. Because there is one more unassigned than assigned edge, this flipping increases the cardinality of the assignment by one, completing a single stage of the algorithm. If an augmenting path is not found, the dual variables are adjusted to bring additional edges into the equality subgraph by making them admissible, and the search continues. n stages of the algorithm are performed to determine the assignment, at which point the algorithm terminates.

However, when we consider the changed cost, in the initialization phase of the dynamic Hungarian algorithm, the affected node is unassigned, decreasing the cardinality of the assignment by one. If a row i^* of the cost matrix changes, the affected node (task) is T_i; whereas if column j^* changed, the affected node (UAV) is U_j.

We modify the dual variables α_i and β_j as follows to add new edges to the equality, as shown in Eq. (12). Let I^* denote the set of subscripts of T_i's in the Hungarian tree, and J^* denote the set of subscripts of U_j's in the Hungarian tree.

$$\theta = \frac{1}{2}\min_{i\in I^*, j\notin J^*}(c_{ij} - \alpha_i - \beta_j),$$
$$\alpha_i = \begin{cases} \alpha_i + \theta & i \in I^* \\ \alpha_i - \theta & i \notin I^* \end{cases}, \tag{12}$$
$$\beta_j = \begin{cases} \beta_j - \theta & j \in J^* \\ \beta_j + \theta & j \notin J^* \end{cases}.$$

Algorithm 2 The dynamic assignment algorithm.

Require: (1) A bipartite graph, $\{V, U, E\}$ (where $|V| = n, |U| = m$) and an $n \times m$ matrix of edge costs; (2) a complete matching A, and the final values of all dual variables α_i and β_j; (3) Either a row c_{i^*} or column c_{j^*} of changed edge costs.

Ensure: A complete assignment.

1: **if** A row i^* of the cost matrix changed: **then**
2: (a) Remove edge (T_{i^*},mate(T_{i^*})) from the assignment A.
3: (b) Assign $\alpha_i = \min_j(c_{i^*j} - \beta_j)$.
4: **if** A column j^* of the cost matrix changed: **then**
5: (a) Remove edge (mate(U_{j^*}),U_{j^*}) from the assignment A.
6: (b) Assign $\beta_j = \min_i(c_{ij^*} - \alpha_i)$.
7: **end if**
8: **end if**
9: Grow the Hungarian trees rooted at unassigned vertices (Task or UAV) in the equality subgraph. If an augmenting path is found, go to step (10), If not, proceed to step (9).
10: Modify the dual variables α_i and β_j by Eq. (12). Then go to step (8).
11: Output the resulting assignment A'.

Fig. 7 illustrates an example of the dynamic Hungarian algorithm in task assignment with dynamic costs. We start off with the optimal solution to the assignment problem shown in Fig. 7(a). In the graph on the left side of this figure, admissible edges are illustrated with dotted lines while assigned edges are shown with solid lines. In the cost matrix on the right, the costs of assigned edges are filled with the pink color.

Suppose now that the costs in the third column of the matrix change as illustrated in Fig. 7(b), the changed costs are filled with the gray color. Then no edges are connected to U_3 in the graph on the left side of this figure. Removing the variable β_3, we need to re-compute the value of β_3, as illustrated in Fig. 7(c).

The search for an augmenting path begins from the only unassigned node T_4. The search fails, requiring modifications of the vertex labels, as shown in Fig. 7(d). The search for an augmenting path then continues, yielding the path (T_4, U_2, T_1, U_3). The current assignment is then augmented along this path by flipping the assigned and unassigned edges, as shown in Fig. 7(e). The final assignment is shown in Fig. 7(f).

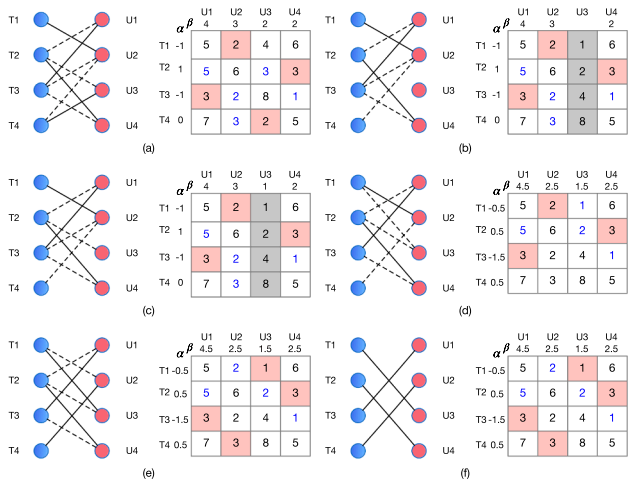


Fig. 7. An example of the dynamic Hungarian algorithm: (a) The optimal solution to the assignment problem; (b) the gray grids represent the cost change in the third column of the matrix; (c) re-compute the value of β_3; (d) the search fails, requiring modifications of the vertex labels; (e) flipping the assigned and unassigned edges; (f) the final assignment result.

5. The SE-ABC algorithm for path planning

The purpose of path planning in a multi-UAV network is to find a sequence of locations that form the shortest path from the starting location to the target location, where each subpath connecting any adjacent locations on this path will not intersect with the known threat/obstacle. We propose a self-evolved artificial bee colony (SE-ABC) algorithm to address the problem, where the bee colony consists of three groups: employed bees, onlooker bees, and scout bees.

5.1. Main procedure of the SE-ABC algorithm

The SE-ABC algorithm follows the main procedure of the conventional ABC algorithm, which employs three groups of bees in three stages: (1) A bee waiting on the dance area for making a decision to choose a food source is called an onlooker; (2) a bee going to the food source visited by itself previously is named an employed bee; and (3) a bee carrying out random search is called a scout [50].

Let $P_i = (P_i^1, P_i^2, \ldots, P_i^{n_{sum}})$ represent a candidate position searched by the ith employed bee. Here, a position refers to a feasible solution for the optimization problem. The number of employed bees SN is equal to that of onlooker bees, and that of food sources. In each iteration, onlooker bees search locally around those qualified employed bees which **are selected** by the roulette selection strategy. Then, those employed bees that cannot improve their positions further in a certain iteration will be replaced with randomly initialized scout bees. The algorithm is typically executed in three stages.

Stage 1 (Employed Bees Phase): At first, the initial positions of food sources are randomly generated by employed bees in the feasible solution space. The process to initially generate the jth element of the ith solution P_i^j is represented as below:

$$P_i^j = P_{min}^j + rand(0,1)\cdot(P_{max}^j - P_{min}^j), \quad (i = 1,2,\ldots,n_{sum};\ j = 1,2,\ldots,SN) \tag{13}$$

where $P_{\max}^j$ and $P_{\min}^j$ denote the lower and upper boundaries of this jth element.

In each iteration, each employed bee executes the crossover and mutation processes to share information with one randomly-chosen neighbor, and refresh their position P_i^{*j} by Eq. (14):

$$P_i^{*j} = P_i^j + rand(-1,1)\cdot(P_q^j - P_i^j), \tag{14}$$

where the ith employed bee exchanges information with the qth employed bee in the jth element. Here, the value of j is randomly selected from 1 to n_{sum}, and the value of q is randomly selected from 1 to SN, while meeting the condition that $q \neq i$. If P_i^{*j} is better than P_i^j, the position is updated; otherwise, the position remains as P_i^j.

Stage 2 (Onlooker Bees Phase): Then, each of the onlooker bees needs an employed bee to follow. Each onlooker bee randomly chooses to exploit or not around the followed employed bee's position with the probability of η_i, which is calculated by Eq. (15):

$$\eta_i = \frac{fit(i)}{\sum_{j=1}^{SN} fit(j)}, \mathbb{ZZ}$$
$$fit(i) = \frac{1}{1 + obj(i)}. \tag{15}$$

Here, $obj(i)$ is the objective function value of the ith nectar source. Apparently, the lower $obj(i)$ is, the higher probability for the ith nectar source to be selected by onlooker bees, which is known as the roulette selection strategy.

Stage 3 (Scout Bees Phase): Once an onlooker bee exploits around an employed bee without finding a better position, the invalid trial counter $trail$ is incremented by one; otherwise, the corresponding $trail$ is set to zero instantly. In other words, when $trail$ exceeds a certain threshold D, the employed bee will be abandoned and directly replaced by a scout bee with a randomly initialized position by Eq. (13).

When the maximum number of iterations $MaxCycle$ of the artificial bee colony algorithm is reached, the search process for the optimal path is completed.

5.2. Three mechanisms in the SE-ABC algorithm

The SE-ABC algorithm mainly differs from the conventional ABC algorithm in three aspects:

- In the stage of employed bee search, a self-adaptive optimal solution guidance strategy is adopted to balance the capabilities of exploitation and exploration.
- In the stage of onlooker bee search, to avoid premature convergence, the SE-ABC method applies the Boltzmann selection mechanism [29] instead of the roulette strategy.
- **Thirdly**, a chaotic local search operator is used to improve the ability of local search and to prevent falling into the local optimal solution.

5.2.1. Balancing exploitation and exploration

The capability of exploitation refers to the ability to search within a specific area and extract good solutions; and the capability of exploration refers **to** the ability of exploring different areas of the search space in order to determine a better solution. The conventional ABC algorithm focuses on improving the exploration capability, but at the cost of sacrificing the exploitation capability, which can easily lead to slow convergence speed and low solution accuracy.

Inspired by the differential thinking in the Differential Evolution (DE) algorithm [51], an improved search equation is proposed in the SE-ABC algorithm:

$$P_i^{*j} = P_{best}^j + rand(-1,1) \cdot (P_{q_1}^j - P_{q_2}^j). \tag{16}$$

Here, P_{best}^j is the global optimal solution for the current iteration. In the above search equation, a new search area is explored around the optimal solution obtained in the last iteration. It uses the optimal solution to guide and randomly select two individuals (P_{q_1} and P_{q_2}) to explore the best possible area, thereby improving their exploitation capability.

Although this search method improves the development ability to a certain extent, it does not fully consider the problem that the exploration capability needs to be strong in the early stage of the cycle, while high exploitation capability needs to be maintained in the later iterations. Therefore, this paper proposes a search Eq. (17) for the self-adaptive optimal solution guidance strategy:

$$P_i'^{*j} = \xi \cdot P_i^j + (1-\xi) \cdot P_{best}^j + rand(-1,1) \cdot (P_{q_1}^j - P_{q_2}^j),$$
$$\xi = 1 - rand^{(1-t/MaxCycle)^2}. \tag{17}$$

Here, t is the current number of iterations (replacing 'cycle' to avoid confusion), $MaxCycle$ is the maximum number of iterations, and ξ is the self-adaptive coefficient that determines the dependence degree of the nectar search Eq. (14) and the optimal solution guidance search Eq. (16) on the number of iterations.

In the early stage of the iteration, $\xi \approx 1$, so Eq. (14) is mainly used to maintain a strong capability of exploration. In other words, this improves the global search capabilities and avoids falling into local optima. As the number of iterations increases, ξ gradually approaches 0, and Eq. (16) becomes dominant to meet the refined search needs around the optimal solution. This improves the convergence speed and the accuracy of the solution.

With the above improvements, the SE-ABC algorithm is able to effectively balance and make full use of exploration and exploitation capabilities.

5.2.2. Boltzmann selection for onlooker bee search

In the conventional ABC algorithm, onlooker bees choose the food source according to the probability defined by the roulette strategy. However, the roulette method easily leads to a decline in population diversity and causes the algorithm to converge prematurely.

In contrast, the Boltzmann selection strategy has inherent parallelism, robustness, and stability, and is widely used in machine learning, adaptive control, and other fields. Theoretically, the Boltzmann selection mechanism can effectively solve the local optimal problem. Hence, the SE-ABC algorithm adopts the Boltzmann selection strategy to replace the roulette strategy. Eq. (18) is designed for probability selection and replaces Eq. (15) in the conventional ABC:

$$\eta_i' = \frac{e^{fit(i)/T}}{\sum_{j=1}^{SN} e^{fit(j)/T}}, \mathbb{Z}$$
$$T = T_0 \cdot (0.99^{(t-1)}), \tag{18}$$

where t is the current iteration number, T is the temperature, and T_0 is the initial temperature. Inspired by the simulated annealing algorithm, T decreases as the number of iterations increases. Different stages require different selection pressures: in the early stage, there is less pressure to choose, allowing poorer individuals a chance of survival to maintain high population diversity; in the later stage, there is more pressure to narrow the search field and speed up convergence near the optimal solution.

5.2.3. The use of chaotic local search operator

Thirdly, the conventional ABC algorithm has a weak local search ability. When each generation searches around the best individual P_{best}, it is easy to fall into a local optimal solution.

After the first two stages (employed bees and onlooker bees), the algorithm uses a chaotic local search operator as defined in Eq. (19). The basic idea of this operator is to use the randomness, ergodicity, and regularity of chaotic sequences.

$$y_{k+1} = 4 \times y_k(1 - y_k), \quad (k = 1, 2, 3 \ldots, K) \tag{19}$$

where $y_k \in (0,1)$, $y_k \neq \{0.25, 0.5, 0.75\}$, and K is the length of the chaotic sequence. Note that the Logistic chaos sequence used in this paper is the most commonly used chaos sequence [52].

We propose a new optimization strategy. Firstly, food sources of each iteration are sorted according to the objective function value (the smaller the objective function value, the better the food source), and we select the top $SN/2$ food sources. Then, the algorithm randomly selects a food source P_m ($m = 1, 2, \ldots, SN/2$) as the mth suboptimal solution.

A local search is performed around P_m, and the individual $P'_m(K)$ after K chaotic iterations is greedily selected against the corresponding food source P_{best} of this iteration. Here, y_k is calculated by Eq. (19), and the local search is performed by the following equation:

$$P'_m(k) = (1-\tau)P_m(k) + \tau \cdot y_k, \quad \tau = \frac{MaxCycle/2 - t + 1}{MaxCycle/2}, \tag{20}$$

where τ is the shrinkage factor, which decreases as the number of iterations t increases. Here, we only select the local search duration as $MaxCycle/2$ to prevent falling into local optima and premature convergence in the early stage of evolution. In the later stage of evolution, there is no need to expand the scope of local search, as the algorithm only needs to search near the optimal solution. The pseudo-code of the SE-ABC algorithm is given by Algorithm 3.

Algorithm 3 The SE-ABC algorithm.

Require: Population size SN, Max iterations $MaxCycle$, Limit D.
Ensure: Optimal solution P_{best}.

Set $trail_i = 0$ for all food sources.
Randomly initialize locations P_i by Eq. (13).
for $t = 1 : MaxCycle$ **do**
  // **Employed Bees Phase**
  for $i = 1 : SN$ **do**
    Produce new solution $P_i^{*'}$ by using Eq. (17).
    Calculate fitness $fit(P_i^{*'})$.
    if $fit(P_i^{*'}) > fit(P_i)$ **then**
      $P_i = P_i^{*'}$; $trail_i = 0$.
      $trail_i = trail_i + 1$.
    end if
  end for
  // **Onlooker Bees Phase**
  Calculate selection probability η'_i using Eq. (18).
  for $i = 1 : SN$ **do**
    Select a solution P_i based on η'_i.
    Perform search and greedy selection (same as Employed Phase).
  end for
  // **Chaotic Local Search Phase**
  for $i = 1 : SN/2$ **do**
    Generate chaotic sequence y_k by Eq. (19).
    Randomly select P_m, generate P'_m using Eq. (20).
    Apply greedy selection between P'_m and P_{best}.
  end for
  // **Scout Bees Phase**
  for $i = 1 : SN$ **do**
    if $trail_i > D$ **then**
      Replace P_i with new random solution by Eq. (13).
      $trail_i = 0$.
    end if
  end for
end for
Output optimal solution P_{best}.

6. Performance evaluation

In this section, we compare the proposed SE-ABC algorithm with existing approaches for path planning through extensive experiments. We implement the SE-ABC algorithm in a prototype system with DJI Phantom 3 quadcopters and conduct experiments in a typical 3D scenario on campus to complete a mission of environmental monitoring and sampling.



Fig. 8. Prototyped on the DJI Phantom 3 quadcopter.

6.1. Experiment setup

Prototype: Multiple UAVs (DJI Phantom 3 quadcopters, as shown in Fig. 8) form the network [53]. Each UAV provides an API for trajectory configuration, and its onboard GPS sensor reports real-time 3D location information. Given the start/target locations and threat positions, we compute the flight path for each UAV offline and upload the generated waypoints via an intelligent flight mode, enabling autonomous execution under GPS guidance. An HD camera or environmental monitoring sensors are mounted underneath the UAV to capture environmental signals while hovering at designated locations. In our experimental scenario, threats are modeled as obstacles (i.e., buildings with known location contours). All algorithms are implemented in Python on a standard PC. Table 3 summarizes the prototype setup, sensors, computing platform, and software tools for reproducibility. Moreover, we focus on scenarios with known static threats/obstacles, which is a common assumption in offline mission planning and enables a clear evaluation of the proposed hierarchical optimization framework.

Compared algorithms: We compare the proposed SE-ABC algorithm with the conventional Artificial Bee Colony (ABC) algorithm, the Particle Swarm Optimization (PSO) algorithm, and the Genetic Algorithm (GA). Since SE-ABC is designed as a training-free swarm intelligence metaheuristic, benchmarking it against standard metaheuristics under the same mathematical formulation provides a rigorous baseline. This approach directly isolates and validates the performance improvements yielded by our proposed self-adaptive operators and chaotic search mechanisms.

Performance metrics: We use the following three metrics for evaluating the performance of the compared algorithms:

- *Cost*: The cost value is defined as the weighted sum of the threat cost and the power consumption, which is the primary metric used to determine whether the path is the optimal solution.
- *Path length*: The path distance is defined as the sum of the Euclidean lengths of all sub-paths connecting adjacent waypoints in the trajectory.
- *Time consumed*: The time consumed is defined as the total duration required for the UAV to complete the flight trajectory and perform measurement tasks at designated hover points.

6.2. Main experimental results

6.2.1. Cost value

In Fig. 9, we compare the convergence of the cost value among the algorithms. It is observed that the SE-ABC algorithm maintains the lowest cost value across different numbers of iterations. In each curve, the best cost value decreases as the number of iterations increases. Compared with the GA and standard ABC algorithms, the PSO algorithm converges faster in the early stage but tends to fall into local optima.

In contrast, the proposed SE-ABC algorithm fully considers the early exploration capability and the later exploitation capability in each evolutionary stage, effectively preventing premature convergence. As

Table 3
Prototype experimental setup for reproducibility.

Item	Details
UAV platform	DJI Phantom 3 quadcopter(s); prototype multi-UAV network.
Sensors	GPS (real-time 3D location); HD camera / environmental monitoring sensors (mounted underneath).
Trajectory execution	Offline planning; waypoint upload; intelligent flight mode for autonomous flight.
Threat definition	Obstacles (buildings) with known location contours.
Computing platform	Standard PC; offline computation.
Software tools	Python; all compared algorithms implemented.

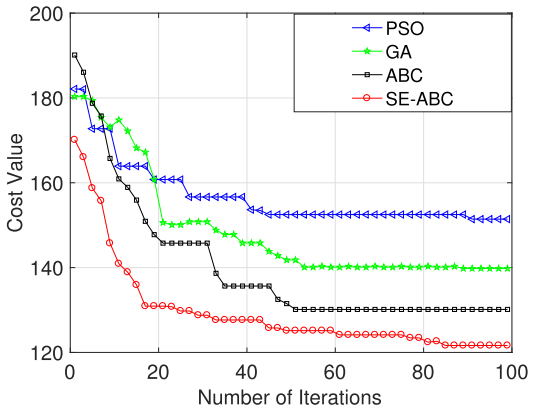


Fig. 9. The cost value of SE-ABC compared with PSO, GA, and ABC for 3D path planning, showing a lower final cost and more stable convergence.

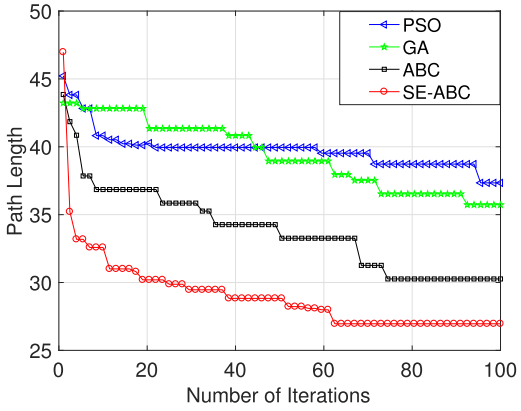


Fig. 10. The path length under the compared algorithms, indicating that SE-ABC achieves shorter safe trajectories in most cases.

shown in the figure, SE-ABC achieves a lower final cost. Moreover, when the number of iterations exceeds 40, the cost value decreases only marginally, indicating that increasing the number of iterations further is not sufficient to find a better path.

6.2.2. *Path length*

Fig. 10 shows the path length generated by different algorithms. Note that the path planning algorithm considers not only the path length but also flight safety (distance to threats, i.e., buildings). Hence, the optimal path (minimum cost) is not necessarily the shortest path in terms of Euclidean distance. It is feasible that the total cost of the planned path is the lowest, yet the path length is not the shortest, which aligns with the objective of this study. For example, between iterations 20 and 40, although the path cost of GA is lower than that of PSO (see Fig. 9), the path length of GA is greater than that of PSO (see Fig. 10). However, in our experiments, the path found by SE-ABC consistently achieves the shortest length among valid safe paths in most cases.

6.2.3. *Time consumed*

We conduct a measurement study on the total time consumed to perform a mission, which includes both flight time and measurement time at each location in the path.

Impact of different measurement times: In Fig. 11, we plot the time consumed under different measurement granularities. The path planning is performed offline, generating a path of N_{sum} waypoints. At every location, the UAV hovers for $t_m \in \{0, 1, 2, 5\}$ s to monitor the environment. From Fig. 11, we observe that our SE-ABC algorithm consumes less total time than the other three algorithms in all cases.

Different distances between starting and target locations: We tested three pairs of starting and target locations with varying distances. Fig. 12 shows that SE-ABC consumes the least time. Interestingly, although the distance increases exponentially (100, 200, 400 m), the time consumed does not strictly follow an exponential growth. This is attributed to SE-ABC's capability to intelligently detour around threats, meaning the flight time depends on obstacle distribution rather than linear distance alone.

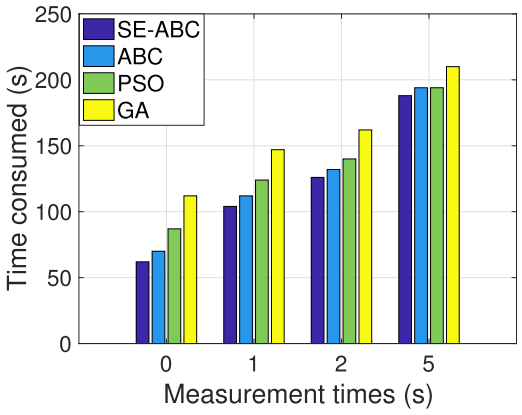


Fig. 11. Time consumed of three algorithms with different granularity of measurement time.

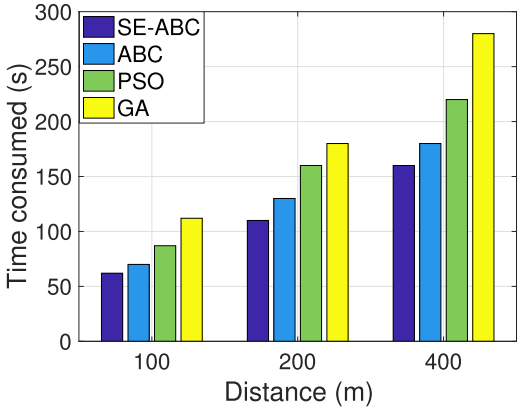


Fig. 12. Time consumed vs. distance between target and starting location.

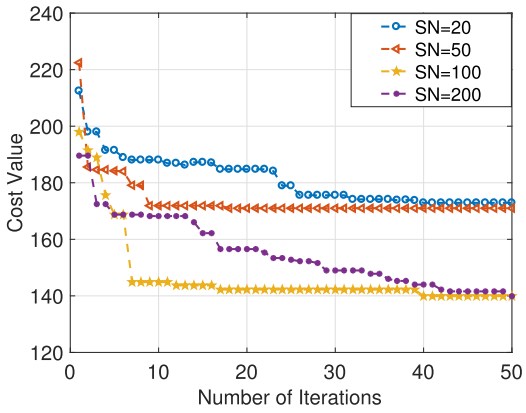


Fig. 13. The impact of parameters SN and $MaxCycle$ on the performance ($SN \in \{20, 50, 100, 200\}$, $MaxCycle = 50$).

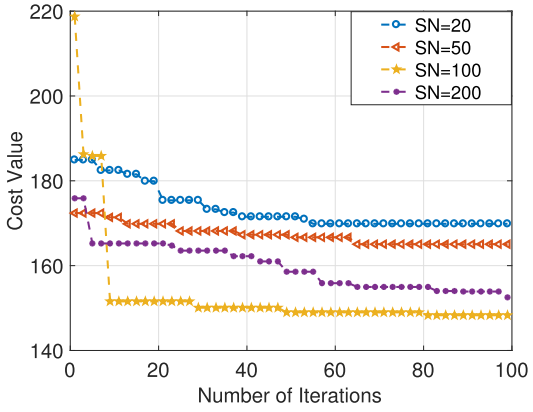


Fig. 14. The impact of parameters SN and $MaxCycle$ on the performance ($SN \in \{20, 50, 100, 200\}$, $MaxCycle = 100$).

6.2.4. Impact of two parameters in the SE-ABC algorithm

In Fig. 13, we analyze the impact of population size (SN) on performance. Theoretically, SN significantly affects the cost value and should be neither too large nor too small. Fig. 13 shows that the cost value with $SN = 20$ or 50 is higher than with $SN = 100$ or 200. However, after 50 iterations, the results for $SN = 100$ and $SN = 200$ are very close. This implies that a moderate population size is sufficient.

In Fig. 14, by doubling $MaxCycle$, we observe that the cost value tends to remain unchanged. This implies that the SE-ABC algorithm has converged to the optimal solution, and more iterations will increase computation overhead without further improving performance. Therefore, we recommend setting SN between 50 and 100, and $MaxCycle$ between 100 and 200.

6.2.5. Tradeoff between planning a safe path and a shortest one

In typical 3D scenarios, there is a tradeoff between safety (distance from threats) and energy efficiency (path length). This is controlled by the weight γ. If $\gamma \to 0$, a shorter but riskier path is selected. If $\gamma \to 1$, a safer but longer path is chosen. Fig. 15 shows this impact: when $\gamma \approx 0.52$, the path length is about 35, and the probability of an unsafe path reduces to 39%, striking a balance. Hence, the selection of γ critically impacts safety and cost. In our experiments, we use $\gamma = 0.5$ as a balanced default to jointly account for safety and efficiency. Fig. 15 also serves as a sensitivity view by varying γ, illustrating how the planner shifts between shorter but riskier paths and safer but longer trajectories. In practice, γ can be tuned according to mission priorities (e.g., larger γ for safety-critical missions or threat-dense environments, and smaller γ for efficiency-oriented missions in sparse-threat

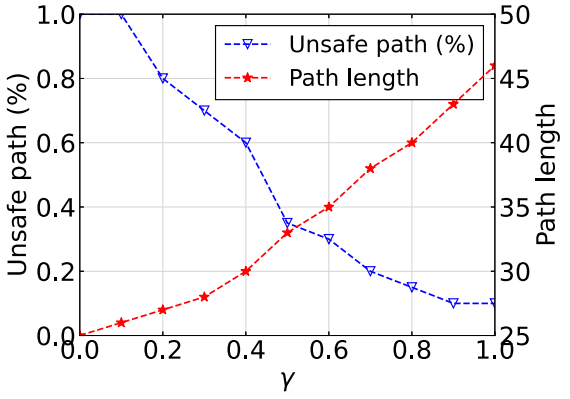


Fig. 15. Tradeoff between planning a safe path and a shortest path.

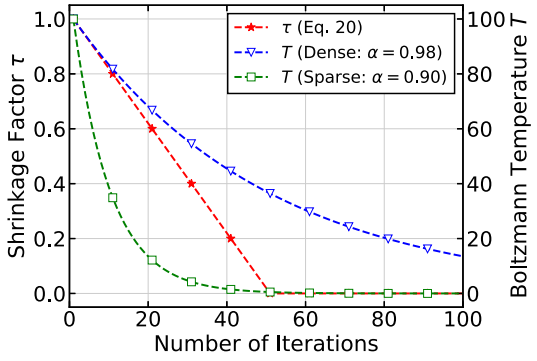


Fig. 16. Parameter sensitivity analysis: the dynamic adaptation of the chaotic shrinkage factor (τ) and the Boltzmann temperature decay (T) under different environmental topologies.

6.2.6. Sensitivity of temperature decay and shrinkage factor

Furthermore, to provide actionable deployment guidelines, we investigated the sensitivity of the Boltzmann temperature decay rate (α) and the chaotic local search shrinkage factor (τ). These parameters critically balance the algorithm's exploration and exploitation capabilities. In environments with dense threats, a slower temperature decay (e.g., $\alpha = 0.98$) and a gradually decreasing τ are recommended to maintain population diversity and avoid premature convergence into local optima. Conversely, in sparse environments, a faster decay (e.g., $\alpha = 0.90$) and a rapidly shrinking τ accelerate convergence toward the optimal trajectory. The dynamic adaptation curves under these different topologies are illustrated in Fig. 16.

6.3. Simulation results

We simulate the multi-UAV network in a cubic space of $21 \times 21 \times 2$ km containing multiple threats (peaks with different heights).

6.3.1. Symmetric assignment and path planning

In this part, we assign n tasks to n UAVs with the least total cost. Fig. 17 shows a path found by a UAV avoiding threats. We simulate three UAVs with battery levels of 98%, 70%, and 40%, respectively. They start from the same (or different) locations to execute three tasks. The planned paths are shown in Figs. 18 and 19. The results confirm that the UAVs successfully avoid threats and reach target locations under the symmetric assignment paradigm.

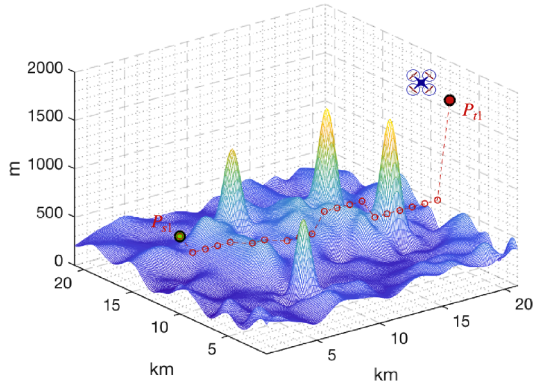


Fig. 17. Single-UAV path planning in a 3D scenario, illustrating threat avoidance along the planned trajectory.

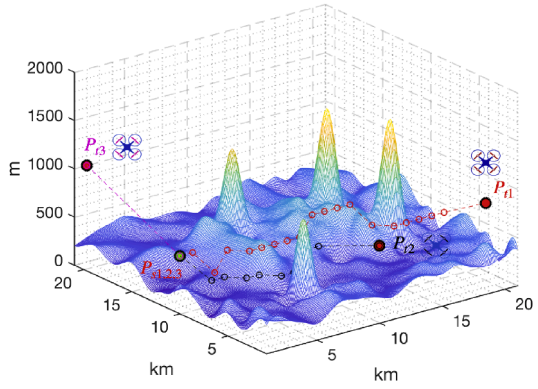


Fig. 18. Multi-UAV path planning with symmetric assignment from a common start, showing coordinated threat-avoiding trajectories.

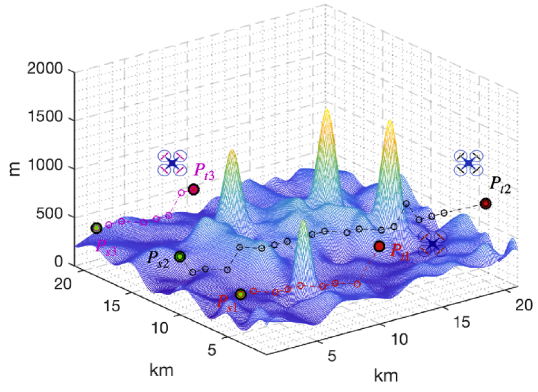


Fig. 19. Multi-UAV path planning with symmetric assignment from different starting locations, showing consistent threat avoidance across heterogeneous starts.

6.3.2. *Asymmetric assignment and path planning with dynamic costs*

In this scenario, we simulate three UAVs performing three tasks. During the mission, UAV 2 suffers sudden battery damage and is removed from the system. The remaining two UAVs must take over the tasks. Since UAV 1 has more remaining battery life than UAV 3, the dynamic assignment algorithm determines that UAV 1 is more appropriate to take over the task. As shown in Fig. 20, UAV 1 first completes its originally assigned task and then proceeds to the second target location T_2. This confirms that our approach is robust to dynamic cost changes and node failures.

Furthermore, to quantitatively evaluate the scalability and real-time performance of the proposed framework, we measured the computational overhead of the dynamic task assignment layer (Algorithm 2) under varying network scales. Due to the practical limitations of large-scale physical deployment, these tests were performed in a simulation environment on a standard PC to verify the algorithm's capability. As shown in Table 4, the execution time scales with the augmented matrix size $k = \max(n, m)$, operating at a complexity of $\mathcal{O}(k^3)$. Remarkably, even in a highly dense scenario with 200 UAVs and 200 tasks, the dynamic reassignment is computed in merely 1.08 ms. This minimal computational overhead confirms that the assignment layer can seamlessly support real-time, large-scale multi-UAV operations without introducing latency bottlenecks.

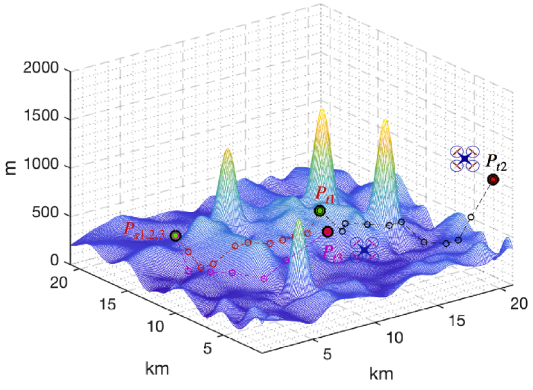


Fig. 20. Multi-UAV path planning with asymmetric assignment and dynamic costs. After UAV 2 fails, UAV 1 takes over and reaches T_2, demonstrating battery-triggered reassignment robustness.

Table 4
Computational overhead of the dynamic task assignment algorithm across varying network scales.

UAVs	Tasks	Matrix Size	Execution Time (ms)
10	10	10×10	0.003
20	30	30×30	0.014
50	50	50×50	0.062
100	120	120×120	0.199
200	200	200×200	1.077

7. Conclusions

In this paper, we have jointly optimized the multi-UAV sensing problem, given a number of known threats (obstacles) in the 3D space, by decoupling the problem into task assignment and path planning subproblems. We proposed an improved Hungarian algorithm and a self-evolved artificial bee colony (SE-ABC) algorithm to address the two subproblems, respectively.

We implemented the proposed algorithms using the DJI Phantom 3 quadcopter platform and verified them through simulations. The experimental results show that the proposed algorithms outperform existing approaches, specifically by lowering the average cost by 15% (including the safety cost given the known threats), shortening the length of the found paths by 23%, and reducing the time consumed in completing multiple sensing tasks by 25%.

While the current formulation assumes that threats (obstacles) in the 3D space are known and the evaluation is conducted under the considered system scale and experimental settings, the scalability to very large UAV swarms and the robustness under highly dynamic threat conditions are not fully explored in this work. Future work will focus on extending the framework to handle dynamic external threats and moving targets. Specifically, we plan to integrate real-time sensor data to dynamically update the threat degree parameter D_k within our event-triggered loop, and implement delay-tolerant mechanisms in the manager layer to mit-

igate communication latency. Further investigations will also address the framework's scalability under larger-scale multi-UAV settings.

CRediT authorship contribution statement

Yijun Lu: Conceptualization, Data curation, Formal analysis, Funding acquisition, Investigation, Methodology, Validation, Visualization, Writing – original draft; **Yufang Liu:** Data curation, Validation; **Yuyin Ma:** Formal analysis, Writing – original draft; **Tingting Xiao:** Validation, Writing – review & editing; **Pengpeng Qiao:** Methodology, Project administration, Resources, Supervision, Validation.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Yijun Lu reports financial support was provided by Japan Science and Technology Agency. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

References

[1] E. Yanmaz, H.M. Balanji, İ. Güven, Dynamic multi-UAV path planning for multi-target search and connectivity, IEEE Trans. Veh. Technol. 73 (7) (2024) 10516–10528.

[2] Q. Fan, F. Wang, X. Shen, D. Luo, Path planning for a reconnaissance UAV in uncertain environment, in: Control and Automation (ICCA), 2016 12th IEEE International Conference on, IEEE, 2016, pp. 248–252.

[3] D.L. Newman, Drone for Collecting Images and System for Categorizing Image Data, 2014, (U.S. Patent Application 13/998,871).

[4] S. Alqefari, M.E.B. Menai, Multi-UAV task assignment in dynamic environments: current trends and future directions, Drones 9 (1) (2025) 75.

[5] P. Zhao, Y. Yang, Y. Zhang, K. Bian, L. Song, P. Qiao, Z. Li, Optimal trajectory planning of drones for 3D mobile sensing, in: 2018 IEEE Global Communications Conference (GLOBECOM), IEEE, 2018, pp. 1–6.

[6] F.Y. Vincent, S.-W. Lin, Multi-start simulated annealing heuristic for the location routing problem with simultaneous pickup and delivery, Appl. Soft Comput. 24 (2014) 284–290.

[7] H. Lei, Y. Yan, Q. Han, Optimized particle swarm-ant colony fusion for UAV path planning research, in: 2024 4th International Conference on Artificial Intelligence, Virtual Reality and Visualization, IEEE, 2024, pp. 49–54.

[8] M. Li, P.-F. Sun, S.-W. Jeon, X. Zhao, H. Jin, Multi-uav path planning with genetic algorithm, in: 2023 14th International Conference on Information and Communication Technology Convergence (ICTC), IEEE, 2023, pp. 918–920.

[9] H.W. Kuhn, The hungarian method for the assignment problem, Nav. Res. Logist. 2 (1-2) (1955) 83–97.

[10] J. Munkres, Algorithms for the assignment and transportation problems, J. Soc. Ind. Appl. Math. 5 (1) (1957) 32–38.

[11] W. Wu, Z. Lu, Y. Sun, M.H. Ang, C. Gong, Dynamic task assignment of multiple heterogeneous autonomous aerial vehicles based on consensus with uncertainties, IEEE Aerosp. Electron. Syst. Mag. 40 (3) (2024) 48–60.

[12] J. Wei, Y. Gu, K.L.E. Law, N. Cheong, Adaptive position updating particle swarm optimization for UAV path planning, in: 2024 22nd International Symposium on Modeling and Optimization in Mobile, Ad Hoc, and Wireless Networks (WIOPT), IEEE, 2024, pp. 124–131.

[13] Z.-c. Wang, T.-l. Xu, F. Liu, Y.-p. Wei, Artificial bee colony based optimization algorithm and its application on multi-drone path planning, AIP Adv. 15 (5) (2025).

[14] X. Gu, G. Zhang, A survey on UAV-assisted wireless communications: recent advances and future trends, Comput. Commun. 208 (2023) 44–78.

[15] S.M.A. Huda, S. Moh, Survey on computation offloading in UAV-enabled mobile edge computing, J. Netw. Comput. Appl. 201 (2022) 103341.

[16] I. Chen, H. Boyer, J.M.-J. Hsu, F.B.A. Gaugel, D. Bauk, A. Schoter, C. Damm, J. Wu, Automated drone systems, 2022, (U.S. Patent 11,215,986).

[17] H. Hernandez, E.W. Jackson, R. Linchangco, UAV surveying system and methods, 2024, (U.S. Patent 11,922,620).

[18] Z. Liu, J. Guo, F. Huang, D. Cai, Y. Wu, X. Chen, K.K. Igorevich, Lightweight trustworthy message exchange in unmanned aerial vehicle networks, IEEE Trans. Intell. Transp. Syst. 24 (2) (2021) 2144–2157.

[19] J. Guo, H. Gao, Z. Liu, F. Huang, J. Zhang, X. Li, J. Ma, ICRA: an intelligent clustering routing approach for UAV Ad Hoc networks, IEEE Trans. Intell. Transp. Syst. 24 (2) (2022) 2447–2460.

[20] Z.A. Çil, H. Öztop, Z.D. Kenger, D. Kizilay, Integrating distributed disassembly line balancing and vehicle routing problem in supply chain: integer programming, constraint programming, and heuristic algorithms, Int. J. Prod. Econ. 265 (2023) 109014.

[21] Y. Wang, X. Wang, Y. Wei, Y. Sun, J. Fan, H. Wang, Two-echelon multi-depot multi-period location-routing problem with pickup and delivery, Comput. Ind. Eng. 182 (2023) 109385.

[22] M. Mavrovouniotis, M.N. Anastasiadou, D. Hadjimitsis, Measuring the performance of ant colony optimization algorithms for the dynamic traveling salesman problem, Algorithms 16 (12) (2023) 545.

[23] V. Petrenko, F. Tebueva, V. Antonov, S. Ryabtsev, A. Sakolchik, D. Satybaldina, Evaluation of the iterative method of task distribution in a swarm of unmanned aerial vehicles in a clustered field of targets, J. King Saud Univ.-Comput. Inf. Sci. 35 (3) (2023) 283–291.

[24] Y. Liu, J. Wu, J. Tang, W. Wang, X. Wang, Scheduling optimisation of multi-type special vehicles in an airport, Transp. B Transp. Dyn. 10 (1) (2022) 954–970.

[25] P. Zhou, Y. Shen, Y. Zheng, Y. Zheng, B. Guo, Y. Du, A comprehensive review of ground support equipment scheduling for aircraft ground handling services, Transp. Res. E Logist. Transp. Rev. 203 (2025) 104341.

[26] H. Chakraa, F. Guérin, E. Leclercq, D. Lefebvre, Optimization techniques for multi-robot task allocation problems: review on the state-of-the-art, Robot. Auton. Syst. 168 (2023) 104492.

[27] J. Mao, S. Shi, X. Wang, H. Li, 3D object detection for autonomous driving: a comprehensive survey, Int. J. Comput. Vis. 131 (8) (2023) 1909–1963.

[28] K. Chen, C. Liu, H. Chen, H. Zhang, W. Li, Z. Zou, Z. Shi, RSPrompter: Learning to prompt for remote sensing instance segmentation based on visual foundation model, IEEE Trans. Geosci. Remote Sens. 62 (2024) 1–17.

[29] Y. Fu, Q. Sun, R. Wennersten, X. Pang, W. Liu, Interactive scheduling optimization of regional multi-agent integrated energy systems considering uncertainties based on game theory, J. Clean. Prod. 449 (2024) 141697.

[30] J. Hu, S. Andradóttir, H. Ayhan, Optimal control of queueing systems with error-prone servers, Stoch. Syst. 15 (3) (2025) 220–251.

[31] X. Dai, Z. Xiao, H. Jiang, J.C.S. Lui, UAV-assisted task offloading in vehicular edge computing networks, IEEE Trans. Mob. Comput. 23 (4) (2023) 2520–2534.

[32] M. Tan, Z. Liu, Y. Wang, C.L.P. Chen, Z. Wu, Learning-based resilient adaptive fuzzy optimal consensus for nonlinear multiagent systems under dos attacks, IEEE Trans. Fuzzy Syst. 32 (7) (2024) 3943–3952.

[33] S.J. Al-Kamil, R. Szabolcsi, Optimizing path planning in mobile robot systems using motion capture technology, Results Eng. 22 (2024) 102043.

[34] S. Fu, K. Li, H. Huang, C. Ma, Q. Fan, Y. Zhu, Red-billed blue magpie optimizer: a novel metaheuristic algorithm for 2D/3D UAV path planning and engineering design problems, Artif. Intell. Rev. 57 (6) (2024) 134.

[35] B. Biskin, J. Fliege, A. Martinez-Sykora, Autonomous navigation of unmanned aerial vehicles (UAVs) for border patrolling: a stochastic framework, IMA J. Manag. Math. 36 (2) (2025) 231–254.

[36] B.-J. Yu, F.-F. Wei, T.-Y. Chen, W.-N. Chen, An improved evolutionary reinforcement learning algorithm for UAV online target tracking, in: Proceedings of the Genetic and Evolutionary Computation Conference Companion, 2024, pp. 323–326.

[37] Q. Tong, Z. Yang, G. Chai, Y. Wang, Z. Qi, F. Wang, K. Yin, Driving state evaluation of intelligent connected vehicles based on centralized multi-source vehicle road collaborative information fusion, Environ. Dev. Sustain. 27 (10) (2022) 24107–24126.

[38] B. Yang, L. Wu, J. Xiong, Y. Zhang, L. Chen, Location and path planning for urban emergency rescue by a hybrid clustering and ant colony algorithm approach, Appl. Soft Comput. 147 (2023) 110783.

[39] L. Wu, X. Huang, J. Cui, C. Liu, W. Xiao, Modified adaptive ant colony optimization algorithm and its application for solving path planning of mobile robot, Expert Syst. Appl. 215 (2023) 119410.

[40] E. Kaya, B. Gorkemli, B. Akay, D. Karaboga, A review on the studies employing artificial bee colony algorithm to solve combinatorial optimization problems, Eng. Appl. Artif. Intell. 115 (2022) 105311.

[41] B. Akay, D. Karaboga, B. Gorkemli, E. Kaya, A survey on the artificial bee colony algorithm variants for binary, integer and mixed integer programming problems, Appl. Soft Comput. 106 (2021) 107351.

[42] S. Sathish Kumar, V.J. Arulkarthick, A. Chakrapani, G. Sasikala, A new hybrid precoding-partial transmit sequence method for PAPR reduction in multicarrier wireless communication systems, Proc. Natl. Acad. Sci. India A Phys. Sci. 94 (2) (2024) 193–199.

[43] J. Chen, X. Cao, P. Yang, M. Xiao, S. Ren, Z. Zhao, D.O. Wu, Deep reinforcement learning based resource allocation in multi-UAV-aided MEC networks, IEEE Trans. Commun. 71 (1) (2022) 296–309.

[44] Z. Dai, Y. Zhang, W. Zhang, X. Luo, Z. He, A multi-agent collaborative environment learning method for UAV deployment and resource allocation, IEEE Trans. Signal Inf. Process. Netw. 8 (2022) 120–130.

[45] L. Xu, X. Cao, W. Du, Y. Li, Cooperative path planning optimization for multiple UAVs with communication constraints, Knowl.-Based Syst. 260 (2023) 110164.

[46] K. Katona, H.A. Neamah, P. Korondi, Obstacle avoidance and path planning methods for autonomous navigation of mobile robot, Sensors 24 (11) (2024) 3573.

[47] J. Xu, Q. Sun, Q.-L. Han, Y. Tang, When embodied AI meets industry 5.0: human-centered smart manufacturing, IEEE/CAA J. Autom. Sin. 12 (3) (2025) 485–501.

[48] M. Ershad, A. Sadollah, Optimal solving of uninhabited combat air vehicle path planning using neural network algorithm, in: Proceedings of 7th International Conference on Harmony Search, Soft Computing and Applications: ICHSA 2022, Springer, 2022, pp. 73–82.

[49] D. Rodić, A.P. Engelbrecht, Social networks in simulated multi-robot environment, Int. J. Intell. Comput. Cybern. 1 (1) (2008) 110–127.

[50] D. Karaboga, B. Gorkemli, C. Ozturk, N. Karaboga, A comprehensive survey: artificial bee colony (ABC) algorithm and applications, Artif. Intell. Rev. 42 (1) (2014) 21–57.
[51] H. Hu, Z. Zhang, T. Wang, X. Peng, Underwater glider 3D path planning with adaptive segments and optimal motion parameters based on improved JADE algorithm, Ocean Eng. 299 (2024) 117377.
[52] U. Erkan, A. Toktas, Q. Lai, 2D hyperchaotic system based on Schaffer function for image encryption, Expert Syst. Appl. 213 (2023) 119076.
[53] S.K. Natarajan, P. Shanmurthy, D. Arockiam, B. Balusamy, S. Selvarajan, Optimized machine learning model for air quality index prediction in major cities in India, Sci. rep. 14 (1) (2024) 6795.